

1)Area

```
import java.util.Scanner;
```

```
class Area
```

```
{
```

```
    public static void main(String[]args)
```

```
    {
```

```
        System.out.println("Roll No is 60");
```

```
        Scanner sc= new Scanner(System.in);
```

```
        System.out.println("Length is:");
```

```
        int len = sc.nextInt();
```

```
        System.out.println("breadth is:");
```

```
        int br = sc.nextInt();
```

```
        int area=len*60*br*60;
```

```
        System.out.println("Area of rectangle is:"+area);
```

```
    }
```

```
}
```

2)Avg

```
import java.util.Scanner;
```

```
class Avg
```

```
{
```

```
    public static void main(String[]args)
```

```
    {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        System.out.println("Enter First No:");
```

```
        double a= sc.nextDouble();
```

```
        System.out.println("Enter Second No:");
```

```
        double b= sc.nextDouble();
```

```
        System.out.println("Enter Third No:");
```

```
        double c= sc.nextDouble();
```

```
        double av=(a+b+c)/3;
```

```
        System.out.println("Avarage of three no:"+av);
```

```
    }
```

```
}
```

3)Bank

```
abstract class Account
```

```
{
```

```
int accountNumber;
```

```
String accountHolder;
```

```
double balance;
```

```
Account(int accountNumber, String accountHolder, double balance)
```

```
{
```

```
this.accountNumber = accountNumber;
```

```
this.accountHolder = accountHolder;
```

```
this.balance = balance;
```

```
}
```

```
abstract void calculateInterest();
```

```
abstract void deposit(double amount);
```

```
abstract void withdraw(double amount);
```

```
void display()
```

```
{
```

```
System.out.println("Account Number: " + accountNumber);
```

```
System.out.println("Account Holder: " + accountHolder);
```

```
System.out.println("Balance: " + balance);
```

```
}
```

```
}
```

```
class SavingsAccount extends Account
{
double minBalance = 1000;
double interestRate = 5.0;

SavingsAccount(int accNo, String name, double balance)
{
super(accNo, name, balance);
}

void deposit(double amount)
{
balance += amount;
System.out.println("Deposited: " + amount);
}

void withdraw(double amount)
{
if (balance - amount >= minBalance)
{
balance -= amount;
System.out.println("Withdrawn: " + amount);
}
else
{
System.out.println("Minimum balance requirement not maintained!");
}
```

```
}  
}
```

```
void calculateInterest()  
{  
    double interest = balance * interestRate / 100;  
    System.out.println("Interest: " + interest);  
}  
}
```

```
class CurrentAccount extends Account  
{  
    double overdraftLimit = 5000;  
    double interestRate = 2.0;
```

```
    CurrentAccount(int accNo, String name, double balance)  
    {  
        super(accNo, name, balance);  
    }
```

```
    void deposit(double amount)  
    {  
        balance += amount;  
        System.out.println("Deposited: " + amount);  
    }
```

```
void withdraw(double amount)
{
    if (balance + overdraftLimit >= amount)
    {
        balance -= amount;
        System.out.println("Withdrawn: " + amount);
    }
    else
    {
        System.out.println("Overdraft limit exceeded!");
    }
}
```

```
void calculateInterest()
{
    double interest = balance * interestRate / 100;
    System.out.println("Interest: " + interest);
}
}
```

```
public class Bank
{
    public static void main(String[] args)
    {
        SavingsAccount sa = new SavingsAccount(101, "Yash", 5000);
        sa.deposit(1000);
    }
}
```

```
sa.withdraw(3000);  
sa.calculateInterest();  
sa.display();
```

```
CurrentAccount ca = new CurrentAccount(102, "Ketan", 2000);  
ca.deposit(2000);  
ca.withdraw(6000);  
ca.calculateInterest();  
ca.display();  
}  
}
```

4)conversion

```
import java.util.Scanner;
```

```
class Conv
```

```
{
```

```
public static void main(String[] args)
```

```
{
```

```
Scanner sc =new Scanner(System.in);
```

```
{
```

```
int n=sc.nextInt();
```

```
String binary="";
```

```
while(n>0)
```

```
{
```

```
binary=(n%2)+binary;
```

```
n=n/2;
```

```
}
```

```
System.out.println("Binary number is "+binary);
```

```
}
```

```
}
```

```
}
```


5) Employee

```
import java.util.Scanner;
```

```
class Employee {
```

```
    private static int nextId = 1;
```

```
    private int id;
```

```
    private String name;
```

```
    private double salary;
```

```
    private int exp;
```

```
    private String dept;
```

```
    Employee() {
```

```
        id = nextId++;
```

```
        name = null;
```

```
        salary = 0.0;
```

```
        exp = 0;
```

```
        dept = null;
```

```
    }
```

```
    Employee(String n, double s, int e, String d) {
```

```
        id = nextId++;
```

```
        name = n;
```

```
        if (s > 0)
```

```
            salary = s;
```

```
        else
```

```
salary = 0.0;
```

```
exp = e;
```

```
dept = d;
```

```
}
```

```
int getId() {
```

```
return id;
```

```
}
```

```
String getName() {
```

```
return name;
```

```
}
```

```
void setName(String n) {
```

```
name = n;
```

```
}
```

```
double getSalary() {
```

```
return salary;
```

```
}
```

```
void setSalary(double s) {
```

```
if (s > 0)
```

```
salary = s;
```

```
else
```

```
salary = 0.0;
```

```
}
```

```
int getExp() {  
    return exp;  
}
```

```
void setExp(int e) {  
    exp = e;  
}
```

```
String getDept() {  
    return dept;  
}
```

```
void setDept(String d) {  
    dept = d;  
}
```

```
double getYearlySalary() {  
    return salary * 12;  
}
```

```
void applyRaise() {  
    salary = salary + (salary * 0.10);  
}
```

```
void displayEmployeeInfo() {  
    System.out.println("ID: " + id);  
    System.out.println("Name: " + name);  
    System.out.println("Salary: " + salary);  
    System.out.println("Experience: " + exp);  
    System.out.println("Department: " + dept);  
}  
}
```

```
public class EmployeeTest {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);
```

```
        String name, Department;  
        double salary;  
        int experience;
```

```
        System.out.println("Enter details for Employee 1:");  
        name = scanner.nextLine();  
        salary = scanner.nextDouble();  
        experience = scanner.nextInt();  
        scanner.nextLine();  
        Department = scanner.nextLine();
```

```
        Employee employee1 = new Employee(name, salary, experience, Department);
```

```
System.out.println("Enter details for Employee 2:");
```

```
name = scanner.nextLine();
```

```
salary = scanner.nextDouble();
```

```
experience = scanner.nextInt();
```

```
scanner.nextLine();
```

```
Department = scanner.nextLine();
```

```
Employee employee2 = new Employee(name, salary, experience, Department);
```

```
System.out.println("\nYearly Salary of Employee 1 before raise: " +  
employee1.getYearlySalary());
```

```
System.out.println("Yearly Salary of Employee 2 before raise: " +  
employee2.getYearlySalary());
```

```
employee1.applyRaise();
```

```
employee2.applyRaise();
```

```
System.out.println("\nYearly Salary of Employee 1 after 10% raise: " +  
employee1.getYearlySalary());
```

```
System.out.println("Yearly Salary of Employee 2 after 10% raise: " +  
employee2.getYearlySalary());
```

```
System.out.println("\nEmployee 1 Information:");
```

```
employee1.displayEmployeeInfo();
```

```
System.out.println("Employee 2 Information:");
```

```
employee2.displayEmployeeInfo();
```

```
scanner.close();
```

```
}
```

```
}
```

6)fibo

```
import java.util.Scanner;
```

```
class Fib
```

```
{
```

```
public static void main(String[] args)
```

```
{
```

```
int roll=67;
```

```
int a = 0;
```

```
int b = 1;
```

```
while (a <= roll)
```

```
{
```

```
System.out.println(a + " ");
```

```
int next = a + b;
```

```
a = b;
```

```
b = next;
```

```
}
```

```
}
```

```
}
```

7)Grade

```
import java.util.*;
```

```
class Grade
```

```
{
```

```
    public static void main(String[]args)
```

```
    {
```

```
        Scanner sc=new Scanner(System.in);
```

```
        System.out.println("Enter Marks:");
```

```
        int Marks=sc.nextInt();
```

```
        if(Marks>=80)
```

```
        {
```

```
            System.out.println("A");
```

```
        }
```

```
        else if(Marks>=65)
```

```
        {
```

```
            System.out.println("B");
```

```
        }
```

```
        else if(Marks>=50)
```

```
        {
```

```
            System.out.println("C");
```

```
        }
```

```
        else if(Marks>=40)
```

```
        {
```



```
        System.out.println("D");
    }
    else
    {
        System.out.println("Fail");
    }
}
```

8)Guess

```
import java.util.Scanner;
```

```
import java.util.Random;
```

```
class Guess
```

```
{
```

```
    public static void main(String[]args)
```

```
    {
```

```
        Random rm=new Random();
```

```
        Scanner sc=new Scanner(System.in);
```

```
        int rno=rm.nextInt(1,100);
```

```
        int guess;
```

```
        int gcount=0;
```

```
        System.out.println("");
```

```
        while(true)
```

```
        {
```

```
            System.out.println("Enter your guess:");
```

```
            guess=sc.nextInt();
```

```
            gcount++;
```

```
            if(guess<rno)
```

```
            {
```

```
                System.out.println("IS Low");
```

```
}  
else if(guess>rno)  
{  
    System.out.println("Is High");  
}  
else  
{  
    System.out.println("Correct Guess" +gcount+ "guesses");  
    break;  
}  
}  
}  
}
```

9)Jarray

```
import java.util.Scanner;
```

```
public class Jarray
```

```
{
```

```
public static void main(String[] args)
```

```
{
```

```
Scanner sc = new Scanner(System.in);
```

```
System.out.print("Enter number of rows: ");
```

```
int rows = sc.nextInt();
```

```
int[][] jagged = new int[rows][];
```

```
for (int i = 0; i < rows; i++)
```

```
{
```

```
System.out.print("Enter number of elements in row " + (i + 1) + ": ");
```

```
int cols = sc.nextInt();
```

```
jagged[i] = new int[cols];
```

```
System.out.println("Enter " + cols + " elements:");
```

```
for (int j = 0; j < cols; j++)
```

```
{
```

```
jagged[i][j] = sc.nextInt();
```

```
}
```

```
}
```

```
System.out.println("Jagged Array (Matrix Form):");
```

```
for (int i = 0; i < jagged.length; i++)
```

```
{
```

```
for (int j = 0; j < jagged[i].length; j++)
```

```
{  
System.out.print(jagged[i][j] + " ");  
}  
}  
System.out.println();  
}  
}
```

10) Jmenu

```
import java.util.Scanner;
```

```
public class Jmenu
```

```
{
```

```
public static void main(String[] args)
```

```
{
```

```
Scanner sc = new Scanner(System.in);
```

```
int[][] jaggedArray = null;
```

```
int choice;
```

```
do
```

```
{
```

```
System.out.println("MENU");
```

```
System.out.println("1. Create Jagged Array");
```

```
System.out.println("2. Display Elements");
```

```
System.out.println("3. Find Sum of Each Row");
```

```
System.out.println("4. Find Largest Element in Row");
```

```
System.out.println("5. Exit");
```

```
System.out.print("Enter your choice: ");
```

```
choice = sc.nextInt();
```

```
switch (choice)
```

```
{
```

```
case 1:
```

```
    System.out.print("Enter number of rows: ");
```

```
int rows = sc.nextInt();
jaggedArray = new int[rows][];
for (int i = 0; i < rows; i++)
{
    System.out.print("Enter number of columns for row " + i + ": ");
    int cols = sc.nextInt();
    jaggedArray[i] = new int[cols];
    System.out.println("Enter elements for row " + i + ":");
    for (int j = 0; j < cols; j++)
    {
        jaggedArray[i][j] = sc.nextInt();
    }
}
System.out.println("Jagged array created successfully!");
break;
```

case 2:

```
if (jaggedArray == null)
{
    System.out.println("Array not created ");
}
else {
    System.out.println("Jagged Array Elements:");
    for (int i = 0; i < jaggedArray.length; i++)
    {
        for (int j = 0; j < jaggedArray[i].length; j++)
```

```
{  
    System.out.print(jaggedArray[i][j] + " ");  
}  
    System.out.println();  
}  
}  
    break;
```

case 3:

```
if (jaggedArray == null)  
{  
    System.out.println("Array not created");  
}  
else  
{  
    for (int i = 0; i < jaggedArray.length; i++)  
    {  
        int sum = 0;  
  
        for (int j = 0; j < jaggedArray[i].length; j++)  
        {  
            sum += jaggedArray[i][j];  
        }  
        System.out.println("Sum of row " + i + " = " + sum);  
    }  
}
```



```
}
```

```
break;
```

case 4:

```
if (jaggedArray == null)
```

```
{
```

```
System.out.println("Array not created yet!");
```

```
}
```

```
else
```

```
{
```

```
for (int i = 0; i < jaggedArray.length; i++)
```

```
{
```

```
int max = jaggedArray[i][0];
```

```
for (int j = 1; j < jaggedArray[i].length; j++)
```

```
{
```

```
if (jaggedArray[i][j] > max)
```

```
{
```

```
max = jaggedArray[i][j];
```

```
}
```

```
}
```

```
System.out.println("Largest element in row " + i + " = " + max);
```

```
}
```

```
}
```

```
break;
```

```
case 5:
```

```
System.out.println("Exiting program...");
```

```
break;
```

```
default:
```

```
System.out.println("Invalid choice");
```

```
}
```

```
} while (choice != 5);
```

```
}
```

```
}
```

11) LinearQueue

interface Methods

{

void insert(int x);

void delete();

void disp();

}

class LinearQueue implements Methods

{

int queue[] = new int[5];

int front = -1, rear = -1;

public void insert(int x)

{

if(rear == queue.length - 1)

{

System.out.println("Queue Overflow");

}

else

{

if(front == -1)

front = 0;

rear++;

queue[rear] = x;

}

```
}
```

```
public void delete()
```

```
{
```

```
if(front == -1 || front > rear)
```

```
{
```

```
System.out.println("Queue Underflow");
```

```
}
```

```
else
```

```
{
```

```
System.out.println("Deleted element: " + queue[front]);
```

```
front++;
```

```
}
```

```
}
```

```
public void disp()
```

```
{
```

```
if(front == -1 || front > rear)
```

```
{
```

```
System.out.println("Queue is empty");
```

```
}
```

```
else
```

```
{
```

```
for(int i = front; i <= rear; i++)
```

```
{
```

```
System.out.print(queue[i] + " ");
```

```
}  
System.out.println();  
}  
}
```

```
public static void main(String args[])  
{  
    LinearQueue q = new LinearQueue();  
    q.insert(10);  
    q.insert(20);  
    q.insert(30);  
    q.disp();  
    q.delete();  
    q.disp();  
}  
}
```

12)List no.

```
import java.util.*;
```

```
class List
```

```
{
```

```
public static void main(String [] args)
```

```
{
```

```
Set<Integer> number = new HashSet<>();
```

```
number.add(1);
```

```
number.add(2);
```

```
number.add(3);
```

```
number.add(2);
```

```
number.add(1);
```

```
System.out.println("Number" +number);
```

```
}
```

```
}
```

13) NameList

```
import java.util.*;

class NameList
{
    public static void main(String [] args)
    {
        ArrayList<String> name = new ArrayList<>();

        name.add("Raj");
        name.add("kunal");
        name.add("Vedant");
        name.add("Atharv");
        name.add("Yash");

        System.out.println("List of Students:" +name);

        name.remove("Raj");
        name.add("Harshad");

        System.out.println("Student:" +name);
    }
}
```

14) QueueEle

```
import java.util.*;

class QueueEle
{
    public static void main(String[] args)
    {
        Queue<String> name = new LinkedList<>();
        name.add("Yash");
        name.add("Raj");
        name.add("Harshad");
        name.add("Ketan");
        name.add("Walke");
        System.out.println("All Queue: " + name);
        name.remove();
        name.remove();
        System.out.println("Remaining Queue: " + name);
    }
}
```


15) Result

```
class Shape
{
    public String color="red";
    public boolean filled=true;
    Shape(){}
    Shape(String color,boolean filled)
    {
        this.color=color;
        this.filled=filled;
    }
    public String getColor()
    {
        return color;
    }
    public void setColor(String color)
    {
        this.color=color;
    }
    public boolean isFilled()
    {
        return filled;
    }
    public double getArea()
    {
        return 0;
    }
}
```

```
}  
public double getPerimeter()  
{  
    return 0;  
}  
}
```

```
class Circle extends Shape  
{  
    double radius=1.0;  
    Circle(){}  
    Circle(double radius)  
    {  
        this.radius=radius;  
    }  
    Circle(double radius,String color,boolean filled)  
    {  
        super(color,filled);  
        this.radius=radius;  
    }  
    double getRadius()  
    {  
        return radius;  
    }  
    void setRadius(double radius)  
    {
```

```
this.radius=radius;
}
public double getArea()
{
return 3.14*radius*radius;
}
public double getPerimeter()
{
return 2*3.14*radius;
}
}
```

```
class Rectangle extends Shape
{
double width=1.0;
double length=1.0;
Rectangle(){}
Rectangle(double width,double length)
{
this.width=width;
this.length=length;
}
Rectangle(double width,double length,String color,boolean filled)
{
super(color,filled);
this.width=width;
```

```
this.length=length;
}
double getWidth()
{
return width;
}
void setWidth(double width)
{
this.width=width;
}
double getLength()
{
return length;
}
void setLength(double length)
{
this.length=length;
}
public double getArea()
{
return length*width;
}
public double getPerimeter()
{
return 2*(length+width);
}
```

```
}
```

```
class Resultt
```

```
{
```

```
public static void main(String[] args)
```

```
{
```

```
Shape cir=new Circle(2,"red",true);
```

```
Shape rec=new Rectangle(3,4,"blue",true);
```

```
System.out.println("Circle");
```

```
System.out.println("Radius: "+((Circle)cir).getRadius());
```

```
System.out.println("Area of Circle: "+((Circle)cir).getArea());
```

```
System.out.println("Perimeter of Circle: "+((Circle)cir).getPerimeter());
```

```
System.out.println("Rectangle");
```

```
System.out.println("Area of Rectangle: "+((Rectangle)rec).getArea());
```

```
System.out.println("Perimeter of Rectangle:
```

```
"+"((Rectangle)rec).getPerimeter());
```

```
}
```

```
}
```

16) Switch Case

```
import java.util.*;

class Scase
{
    public static void main(String[]args)
    {
        Scanner sc=new Scanner(System.in);

        int choice;

        do
        {
            System.out.println("1.EvenOdd");
            System.out.println("2.Table");
            System.out.println("3.Prime or Not");
            System.out.println("4.Perfect No.or Not");
            System.out.println("5.Exit");
            System.out.println("Select operation");

            choice=sc.nextInt();

            switch(choice)
            {
```

case 1:

```
System.out.println("Enter Number");
int num=sc.nextInt();
if (num % 2 == 0)
{
    System.out.println(" is Even" + num);
}
else
{
    System.out.println(" is Odd"+ num);
}
break;
```

case 2:

```
System.out.println("Enter Number");
int num1=sc.nextInt();
System.out.println("Table"+num1);
for (int i = 1; i <= 10; i++)
{
    System.out.println(num1 + "*" + i + "="+(num1*i));
}
break;
```

case 3:

```
System.out.println("Enter Number");
int num2=sc.nextInt();
int count = 0;
for(int i=1; i < num2; i++)
{
    if (num2 % i == 0)
        count ++;
}
if (count == 2)
{
    System.out.println("Prime number"+num2);
}
else
{
    System.out.println(" not Prime number"+num2);
}
break;
```

case 4:

```
System.out.println("Enter Number");
int num3=sc.nextInt();
int sum = 0;
for(int i=1; i < num3; i++)
{
    if (num3 % i == 0)
```



```
        sum += 1;
    }
    if (sum == num3)
    {
        System.out.println("Prefect number"+num3);
    }
    else
    {
        System.out.println("not Prefect number"+num3);
    }
    break;

    default:
        System.out.println("Invalid Choice");
    }
}
```

```
while(choice!=5);
```

```
}
```

```
}
```

17) ShippingCalculator

```
class ShippingCalculator
{
    private double shippingCost;

    public ShippingCalculator()
    {
        shippingCost = 0.0;
    }

    public void calculateShippingCost(int weightKg)
    {
        shippingCost = weightKg * 5.0;
    }

    public void calculateShippingCost(double weightGrams)
    {
        shippingCost = weightGrams * 0.005;
    }

    public void calculateShippingCost(String category)
    {
        if (category.equalsIgnoreCase("light"))
        {
            shippingCost = 10.0;
        }
        else if (category.equalsIgnoreCase("medium"))
        {

```

```
        shippingCost = 20.0;
    }
    else if (category.equalsIgnoreCase("heavy"))
    {
        shippingCost = 50.0;
    }
    else
    {
        System.out.println("Invalid category!");
        shippingCost = 0.0;
    }
}

public void displayShippingCost()
{
    System.out.println("Shipping Cost: $" + shippingCost);
}

public static void main(String[] args)
{
    ShippingCalculator sc = new ShippingCalculator();

    sc.calculateShippingCost(5);
    sc.displayShippingCost();

    sc.calculateShippingCost(250.0);
    sc.displayShippingCost();
}
```

```
        sc.calculateShippingCost("Light");  
        sc.displayShippingCost();  
    }  
}
```

18)star pattern

```
import java.util.Scanner;
```

```
class Star
```

```
{
```

```
public static void main(String[] args)
```

```
{
```

```
int n = 5;
```

```
for (int i = n; i >= 1; i--)
```

```
{
```

```
for (int p = 0; p < n - i; p++)
```

```
{
```

```
System.out.print(" ");
```

```
}
```

```
for (int st = 1; st <= i; st++)
```

```
{
```

```
System.out.print("*");
```

```
}
```

```
System.out.println();
```

```
}
```

```
for (int i = 2; i <= n; i++)
```

```
{
```

```
for (int p = 0; p < n - i; p++)
```

```
{
```

```
System.out.print(" ");
```

```
}
```

```
for (int st = 1; st <= i; st++)
```

```
{  
System.out.print("*");  
}  
System.out.println();  
}  
}  
}
```

19)Rational

```
import java.util.Scanner;
```

```
class Rational
```

```
{  
    int n;  
    int d;
```

```
    Rational()
```

```
{  
    n = 0;  
    d = 0;  
}
```

```
    Rational(int n, int d)
```

```
{  
    this.n = n;  
    this.d = d;  
}
```

```
    static Rational add(Rational a, Rational b)
```

```
{  
    int num = a.n * b.d + b.n * a.d;  
    int den = a.d * b.d;  
    return new Rational(num, den);  
}
```

```
    static Rational sub(Rational a, Rational b)
```

```
{  
    int num = a.n * b.d - b.n * a.d;  
    int den = a.d * b.d;  
    return new Rational(num, den);  
}
```

static Rational mult(Rational a, Rational b)

```
{  
    int num = a.n * b.n;  
    int den = a.d * b.d;  
    return new Rational(num, den);  
}
```

static Rational div(Rational a, Rational b)

```
{  
    int num = a.n * b.d;  
    int den = a.d * b.n;  
    return new Rational(num, den);  
}
```

void display()

```
{  
    System.out.println(n + "/" + d);  
}  
}
```



```
public class Test
{
    public static void main(String[] args)
    {
        Scanner scanner = new Scanner(System.in);
        int num1, den1;
        int num2, den2;

        System.out.println("Enter numerator and denominator for the first
rational number:");
        num1 = scanner.nextInt();
        den1 = scanner.nextInt();
        Rational r1 = new Rational(num1, den1);

        System.out.println("Enter numerator and denominator for the second
rational number:");
        num2 = scanner.nextInt();
        den2 = scanner.nextInt();
        Rational r2 = new Rational(num2, den2);

        System.out.print("First Rational Number: ");
        r1.display();

        System.out.print("Second Rational Number: ");
```

```
r2.display();
```

```
Rational add = Rational.add(r1, r2);
```

```
System.out.print("Addition of the two rational numbers: ");
```

```
add.display();
```

```
Rational sub = Rational.sub(r1, r2);
```

```
System.out.print("Subtraction of the two rational numbers: ");
```

```
sub.display();
```

```
Rational mul = Rational.mult(r1, r2);
```

```
System.out.print("Multiplication of the two rational numbers: ");
```

```
mul.display();
```

```
Rational div = Rational.div(r1, r2);
```

```
System.out.print("Division of the two rational numbers: ");
```

```
div.display();
```

```
}
```

```
}
```

